
SkillForge

User Personas & UX/Functional Gap Analysis

Product: SkillForge — Open-Source, Browser-Based AI Agent Skill Manager

Document Type: User Research & UX Analysis

Date: March 21, 2026

Version: 1.0

Prepared by: Product & UX Research Team



Alex



Maya



Jordan



Sam



Riley

CONFIDENTIAL — For Internal Use Only

Table of Contents

1. Executive Summary ...	3
2. User Personas ...	4
2.1 Alex the Solo Developer ...	4
2.2 Maya the Team Lead ...	6
2.3 Jordan the Power User / Skill Author ...	8
2.4 Sam the Enterprise Architect ...	10
2.5 Riley the Newcomer ...	12
3. UX/Functional Gap Analysis ...	14
3.1 Onboarding & Discovery Gaps ...	14
3.2 Skill Authoring Gaps ...	14
3.3 Collaboration & Team Gaps ...	15
3.4 Enterprise & Governance Gaps ...	15
3.5 Cross-Agent Compatibility Gaps ...	16
3.6 UX Polish Gaps ...	16
4. Priority Roadmap ...	18
5. Persona–Feature Matrix ...	19

1. Executive Summary

This document presents five user personas and a comprehensive UX/functional gap analysis for SkillForge, an open-source, zero-install, browser-based AI agent skill manager. The analysis is informed by current market research on AI coding agent adoption, developer workflows, and the emerging skill management ecosystem.

Research Context

The AI coding agent landscape is rapidly maturing. Developers now routinely use 2–3 AI agents daily—Cursor for greenfield development, GitHub Copilot for autocomplete, and Claude Code for complex reasoning tasks. A 25-person team study found that Cursor reduced feature tickets by approximately 30%, while GitHub reports a 55% speed improvement with Copilot. However, managing skills (reusable instruction sets) across these agents remains fragmented and manual.

Key Findings

- Growing demand for skill management infrastructure: With 192+ skill libraries on GitHub and an expanding MCP market, agent skill management is transitioning from individual productivity tooling to shared team infrastructure.
- Five distinct user archetypes: Our research identified five personas spanning solo developers, team leads, power users, enterprise architects, and newcomers—each with fundamentally different needs, skill volumes, and governance requirements.
- 23 UX/functional gaps identified: Gaps range from critical onboarding deficiencies for newcomers to missing enterprise governance features. The most impactful gaps center on skill authoring validation, cross-agent compatibility testing, and collaborative workflows.
- CLAUDE.md vs. Skills paradigm gap: Vercel’s research showing 100% success with CLAUDE.md (passive context) versus 56% for skills (active agent decisions) highlights a fundamental design tension that SkillForge must address with better agent format conversion and testing tools.
- Phased roadmap recommended: A four-phase approach is proposed—starting with editor polish and onboarding (serving solo developers and newcomers), then expanding to team collaboration, power-user tooling, and finally enterprise governance.

Personas	Gaps Identified	Critical Gaps	High Gaps	Roadmap Phases
5	23	4	8	4

2. User Personas

The following five personas represent the primary user archetypes for SkillForge, derived from market research, developer interviews, and analysis of current AI agent adoption patterns.

2.1 Alex Torres — The Solo Developer



Alex Torres
Independent Developer / Freelancer
Self-employed, contract work for startups

AGE	EXPERIENCE	LOCATION	PRIMARY TOOLS
28	5 years	Austin, TX	Cursor, Claude Code, VS Code, GitHub

"I just want to write a skill and have it work across my agents without fighting YAML syntax."

Goals

- Quickly create and iterate on custom skills for each client project
- Maintain a personal library of 5–15 reusable skills organized by project type
- Ensure skills work consistently between Cursor and Claude Code
- Minimize time spent on skill maintenance and focus on actual development

Frustrations / Pain Points

- Manually editing YAML frontmatter is error-prone and tedious
- No visual feedback on skill quality—unclear if a skill is well-structured
- Skills that work in Cursor sometimes need rephrasing for Claude Code
- No way to quickly duplicate and customize a skill for a new project

Skill Management Behavior

Alex keeps skills as individual markdown files in a `.cursor/rules/` directory and a `CLAUDE.md` file. When starting a new project, he manually copies skills from a previous project and edits them. He occasionally browses GitHub for community skills but rarely finds ones that match his needs exactly.

SkillForge Usage Pattern

Alex opens SkillForge to create new skills using the wizard and templates, then exports them directly to his project directories. He relies heavily on the split-pane editor for markdown preview and the quality scoring to validate his skills before deployment. Search and filtering help him find skills from past projects to adapt.

Key Features They Care About

- Wizard with templates (Code Reviewer, Documentation Writer)
- Split-pane editor with live markdown preview
- Quality scoring for instant feedback
- Export to agent-specific formats
- Search and filtering across skill library

2.2 Maya Krishnamurthy — The Team Lead



Maya Krishnamurthy
Engineering Manager
25-person seed-stage startup (FinTech)

AGE	EXPERIENCE	LOCATION	PRIMARY TOOLS
34	10 years (3 in management)	San Francisco, CA	Cursor, GitHub Copilot, Slack, Linear, Notion

“Half my team writes great skills, the other half is copy-pasting broken YAML. I need a way to standardize this.”

Goals

- Standardize skill quality and format across the entire engineering team
- Share curated skill collections for common workflows (code review, testing, docs)
- Reduce onboarding time for new hires learning AI agent workflows
- Track which skills the team actually uses and which are abandoned

Frustrations / Pain Points

- Each developer has skills of wildly different quality—no governance or review process
- No way to share a curated set of skills with a new team member
- Collections exist but cannot be shared or exported as a bundle
- No visibility into skill adoption or effectiveness across the team

Skill Management Behavior

Maya maintains a shared Git repo with “approved” skills in a [team-skills/](#) directory. She reviews PRs for new skills but has no automated quality checks. New hires receive a Notion doc with links to skill files, but often struggle to set them up correctly. She runs a monthly “skill review” session to prune outdated skills.

SkillForge Usage Pattern

Maya would use SkillForge’s Collections to organize team-approved skills by workflow. She exports and imports skill bundles when onboarding new developers. She relies on quality scoring to set a minimum quality bar for the team library. The Cross-Agent Sync feature (coming) is her most anticipated feature for keeping Cursor and Copilot skills aligned.

Key Features They Care About

- Collections for organizing and curating team skills
- Export/Import for sharing skill bundles
- Quality scoring to enforce team standards
- Cross-Agent Sync (upcoming)
- Bulk operations for managing the team library

2.3 Jordan Park — The Power User / Skill Author



Jordan Park
Senior Developer & Open-Source Contributor
Mid-size SaaS company (200 employees)

AGE	EXPERIENCE	LOCATION	PRIMARY TOOLS
31	8 years	Seattle, WA	Claude Code, Cursor, GitHub Copilot, Neovim, GitHub Actions

"I maintain 50+ skills with supporting scripts and references. I need real tooling, not a toy editor."

Goals

- Manage a large library of 50+ skills with complex supporting files (scripts, references, assets)
- Publish high-quality skills to the community and receive feedback
- Test skills across multiple agent runtimes to ensure cross-agent compatibility
- Track skill versions and see diffs when updating

Frustrations / Pain Points

- No tooling for testing how a skill performs across different agents
- Manual validation of YAML frontmatter—easy to introduce syntax errors with complex skills
- No version history or diff view for tracking changes to skills over time
- File upload for supporting assets works but lacks organization and preview

Skill Management Behavior

Jordan uses a monorepo for all skills, with each skill having its own directory containing the main skill file plus [scripts/](#), [references/](#), and [assets/](#) subdirectories. He has a CI pipeline that lints YAML and runs basic checks. He publishes skills to a public GitHub repo and tracks stars/issues as a proxy for quality. He frequently tests skills by running them against real agent prompts.

SkillForge Usage Pattern

Jordan uses SkillForge as a power-user authoring environment. He relies on file upload for supporting scripts and references, the quality scoring system for pre-publish validation, and agent compatibility icons to quickly see which agents a skill supports. He's eagerly awaiting the Marketplace feature to distribute skills more broadly.

Key Features They Care About

- Supporting files management (scripts/, references/, assets/)
- Quality scoring and detailed validation
- Agent compatibility indicators (Claude, Cursor, Copilot, Codex)
- YAML frontmatter parsing with error detection
- Marketplace (upcoming) for skill distribution

2.4 Sam Edwards — The Enterprise Architect



Sam Edwards
Principal Engineer / Platform Architect
Fortune 500 financial services company

AGE	EXPERIENCE	LOCATION	PRIMARY TOOLS
42	18 years	New York, NY	GitHub Copilot Enterprise, internal CI/CD, Terraform, Kubernetes

"I can't deploy AI skills to production without audit trails, approval workflows, and SSO. Period."

Goals

- Establish governance for AI agent skills across 500+ developers
- Integrate skill management into existing CI/CD and compliance pipelines
- Generate MCP servers with enterprise-grade authentication and authorization
- Maintain audit trails and approval workflows for all skill changes

Frustrations / Pain Points

- No audit log or change tracking for skill modifications
- No approval workflow—any developer can create and deploy skills without review
- MCP Server generator lacks integration with enterprise SSO (SAML, OIDC)
- No skill signing or verification to ensure integrity of deployed skills

Skill Management Behavior

Sam's organization has a centralized "AI Platform" team that manages approved AI tools. Skills are currently managed through a custom internal system built on top of Git with custom linting hooks. Every skill must pass a security review before deployment. The team evaluates tools based on enterprise requirements: SSO integration, audit logging, RBAC, and API access for automation.

SkillForge Usage Pattern

Sam evaluates SkillForge as a potential front-end for the organization's skill management workflow. The MCP Server generator is of high interest for creating standardized skill endpoints. He would need API access for CI/CD integration, RBAC for team-level permissions, and comprehensive audit logging before recommending adoption. The zero-install browser-based nature is appealing for deployment simplicity.

Key Features They Care About

- MCP Server generator for standardized endpoints
- Export/Import for CI/CD pipeline integration
- Dark mode and keyboard shortcuts (developer experience)
- Zero-install browser-based deployment
- API access (needed but not yet available)

2.5 Riley Nguyen — The Newcomer



Riley Nguyen
Junior Developer
E-commerce startup (Series A, 40 employees)

AGE	EXPERIENCE	LOCATION	PRIMARY TOOLS
23	1 year (just graduated)	Portland, OR	Cursor (just started), VS Code, ChatGPT

"I installed Cursor last week and everyone says I need 'skills' but I don't even know what YAML is."

Goals

- Understand what AI agent skills are and why they matter
- Set up basic skills for their daily work (code review, documentation) using templates
- Learn by exploring demo skills and understanding how they're structured
- Feel confident enough to customize skills after the first week

Frustrations / Pain Points

- Overwhelming number of options and features with no guided starting point
- Doesn't understand YAML frontmatter—what are "globs"? What is "alwaysApply"?
- Template descriptions don't explain when or why to use each one
- No tooltips or contextual help for editor fields and configuration options

Skill Management Behavior

Riley was told to "set up Cursor rules" during onboarding but received minimal guidance. They Googled "Cursor rules examples" and copied a few from blog posts without understanding the structure. They don't use Claude Code or Copilot yet. Their skills are basic and mostly copied from teammates, with minor text edits.

SkillForge Usage Pattern

Riley would enter SkillForge through the wizard, selecting from templates with clear descriptions. Demo skills would serve as learning resources. They would benefit most from an onboarding tour that explains the skill lifecycle (create → edit → test → deploy). Progressive disclosure of advanced features (supporting files, YAML frontmatter, MCP) would prevent overwhelm.

Key Features They Care About

- Wizard with templates and clear descriptions
- Demo skills as learning resources
- Simple, guided UI with tooltips
- Visual skill cards (less intimidating than raw files)
- Search to find skills by keyword

3. UX/Functional Gap Analysis

The following analysis identifies 23 UX and functional gaps across six categories. Each gap is assessed for severity, affected personas, and recommended remediation. Severity levels are defined as:

Critical	Blocks a primary use case for one or more personas	High	Significantly degrades the experience or limits adoption
Medium	Causes friction but has workarounds	Low	Nice-to-have improvement for polish

3.1 Onboarding & Discovery Gaps

ID	Feature / Area	Persona(s)	Severity	Current State	Recommended Fix
OD-01	No in-app onboarding tour	Riley, Alex	Critical	Users land on the skill library with no guided introduction to features or workflow	Add an interactive onboarding tour (5–7 steps) that walks newcomers through creating their first skill
OD-02	Demo skills don't teach progressively	Riley	High	Demo skills are shown as equal-complexity examples with no learning path	Order demo skills by complexity (Beginner → Intermediate → Advanced) with difficulty badges
OD-03	Template descriptions lack context	Riley, Alex	Medium	Template cards show names but don't explain when or why to use each template	Add "Best for:" and "Use when:" descriptions to each template card in the wizard

3.2 Skill Authoring Gaps

ID	Feature / Area	Persona(s)	Severity	Current State	Recommended Fix
SA-01	No live YAML validation	Alex, Jordan, Riley	Critical	YAML frontmatter errors are only caught on save/export, not during editing	Add real-time YAML linting with inline error markers and suggestions as the user types
SA-02	No diff view for skill versions	Jordan, Maya	High	No way to compare current skill content with a previous version	Implement a side-by-side diff view with change highlighting when updating existing skills
SA-03	No skill testing/preview against agents	Jordan, Alex	Critical	No way to simulate how an agent will interpret a skill before deployment	Add a "Test Skill" panel that shows how each supported agent would parse the skill
SA-04	No spell-check or markdown linting	Alex, Riley	Low	Editor relies on browser spell-check only; no markdown formatting validation	Integrate a lightweight markdown linter and surface warnings for common formatting issues

3.3 Collaboration & Team Gaps

ID	Feature / Area	Persona(s)	Severity	Current State	Recommended Fix
CT-01	No shared workspace / team library	Maya, Sam	Critical	Each user has an isolated local library; no way to create a shared team workspace	Implement shareable workspace URLs or a team library with invite-based access
CT-02	No role-based permissions	Maya, Sam	High	All users have the same privileges—no distinction between viewer, editor, and admin roles	Add RBAC with at minimum three roles: Viewer, Editor, Admin
CT-03	Collections cannot be shared	Maya, Alex	High	Collections are local to a single user's browser; no share or export mechanism	Add "Share Collection" functionality with a shareable link or export-as-bundle option
CT-04	No comments or review workflow	Maya, Jordan	High	No way to leave feedback on a skill or request changes before publishing	Add inline commenting and a review/approve workflow for skill submissions

3.4 Enterprise & Governance Gaps

ID	Feature / Area	Persona(s)	Severity	Current State	Recommended Fix
EG-01	No audit log	Sam	High	No record of who created, modified, or deleted skills and when	Implement an immutable audit log capturing all skill lifecycle events with timestamps and user IDs
EG-02	No approval workflow for publishing	Sam, Maya	High	Skills can be created and deployed without any review or approval gate	Add a configurable publish workflow: Draft → Review → Approved → Published
EG-03	No skill signing or verification	Sam	Medium	No cryptographic verification that a deployed skill hasn't been tampered with	Implement skill content hashing and optional signing for integrity verification
EG-04	MCP Server lacks enterprise SSO	Sam	High	Generated MCP server scripts have no authentication—anyone with the URL can access skills	Add SAML/OIDC authentication stubs to generated MCP server scripts

3.5 Cross-Agent Compatibility Gaps

ID	Feature / Area	Persona(s)	Severity	Current State	Recommended Fix
CA-01	No automatic format conversion	Alex, Jordan, Maya	Critical	Skills written for one agent must be manually rewritten for others (different phrasing, structure)	Implement a "Convert to..." feature that adapts skills to each agent's expected format

ID	Feature / Area	Persona(s)	Severity	Current State	Recommended Fix
CA-02	No multi-agent runtime testing	Jordan	High	No way to test how a skill behaves across Cursor, Copilot, Claude Code, and Codex simultaneously	Add a test harness that simulates each agent's skill ingestion and highlights compatibility issues
CA-03	CLAUDE.md vs. Skills paradigm gap	Alex, Maya	Medium	Vercel found 100% success with CLAUDE.md vs. 56% for skills; no guidance on when to use which approach	Add documentation and a decision helper: "Use CLAUDE.md when... Use skills when..."
CA-04	No skill effectiveness metrics	Jordan, Maya	Medium	No data on how often a skill is triggered, ignored, or overridden by agents	Design a lightweight metrics collection mechanism (opt-in) for tracking skill activation rates

3.6 UX Polish Gaps

ID	Feature / Area	Persona(s)	Severity	Current State	Recommended Fix
UX-01	Editor lacks line numbers	Jordan, Alex	Medium	No line numbers in the editor, making it hard to reference specific lines during review	Add toggleable line numbers to the code/markdown editor
UX-02	No drag-and-drop for collections	Maya, Alex	Low	Collection ordering requires manual editing; no drag-and-drop reordering	Implement drag-and-drop reordering for skills within collections and for collection order
UX-03	No swipe gestures on mobile	Riley, Alex	Low	Mobile experience requires tapping small buttons; no gesture-based navigation	Add swipe gestures for common actions: swipe to delete, swipe to archive, swipe between views
UX-04	No undo/redo beyond browser default	Alex, Jordan, Riley	Medium	Editor relies on browser Cmd+Z; no application-level undo/redo with history	Implement application-level undo/redo with a visible history stack
UX-05	Tag autocomplete missing	Alex, Maya	Low	Tags must be typed in full each time; no autocomplete from existing tags	Add tag autocomplete that suggests from all previously used tags across the library
UX-06	Search limited to name/description	Jordan, Maya, Alex	Medium	Search does not index skill content, only metadata fields (name and description)	Extend search to include full-text content indexing with highlighted matches in results

4. Priority Roadmap

Based on the gap analysis, the following four-phase roadmap prioritizes improvements by impact, user reach, and implementation complexity. Each phase targets specific personas and unlocks the next level of adoption.

Phase 1: MVP+ (Editor Polish & Onboarding)

Timeline: 0–3 months | Primary: Alex, Riley

Captures solo developers and newcomers—the largest user segment. Reduces immediate friction and improves first-time experience.

Gap ID	Feature	Severity
OD-01	In-app onboarding tour	Critical
SA-01	Live YAML validation	Critical
OD-02	Progressive demo skills	High
UX-04	Application-level undo/redox	Medium
OD-03	Template descriptions with context	Medium
SA-04	Spell-check and markdown linting	Low

Phase 2: Teams (Collaboration & Sharing)

Timeline: 3–6 months | Primary: Maya, Alex

Unlocks team adoption—the key growth vector. Shared libraries and review workflows address the most requested features for team leads.

Gap ID	Feature	Severity
CT-01	Shared team workspace / library	Critical
CT-03	Shareable collections	High
CT-04	Comments and review workflow	High
CT-02	Role-based permissions	High
CA-01	Cross-agent format conversion	Critical

Phase 3: Power Users (Testing & Metrics)

Timeline: 6–9 months | Primary: Jordan, Maya

Serves the skill author community who drive ecosystem growth. Testing and metrics tools attract contributors and improve overall skill quality.

Gap ID	Feature	Severity
SA-03	Skill testing/preview against agents	Critical
SA-02	Diff view for skill versions	High
CA-02	Multi-agent runtime testing	High

CA-04	Skill effectiveness metrics	Medium
UX-06	Full-text content search	Medium
UX-01	Editor line numbers	Medium

Phase 4: Enterprise (Governance & Compliance)

Timeline: 9–12 months | Primary: Sam, Maya

Enterprise readiness unlocks large-organization adoption and potential monetization. Requires the collaboration foundation from Phase 2.

Gap ID	Feature	Severity
EG-01	Audit logging	High
EG-02	Approval workflows	High
EG-04	MCP Server enterprise SSO	High
EG-03	Skill signing and verification	Medium
CA-03	CLAUDE.md vs. Skills guidance	Medium

5. Persona–Feature Matrix

The following matrix maps feature importance to each persona. High indicates a must-have feature, Medium indicates useful but not blocking, Low indicates marginal relevance, and N/A indicates the feature is not relevant to that persona’s workflow.

Feature	Alex (Solo Dev)	Maya (Team Lead)	Jordan (Power User)	Sam (Enterprise)	Riley (Newcomer)
Wizard & templates	High	Medium	Low	Low	High
Split-pane editor	High	Medium	High	Low	Medium
Quality scoring	High	High	High	Medium	Medium
Visual skill library	High	Medium	High	Low	High
Search & filtering	High	Medium	High	Medium	High
Collections	Medium	High	High	Medium	Low
Export / Import	High	High	High	High	Low
Bulk operations	Low	High	High	High	N/A
Supporting files	Medium	Low	High	Medium	N/A
Agent compatibility icons	High	Medium	High	Medium	Low
YAML frontmatter parsing	High	Medium	High	Low	Low
Dark mode	Medium	Low	High	Low	Medium
Keyboard shortcuts	High	Low	High	Low	Low
MCP Server generator	Medium	Medium	High	High	N/A
Cross-Agent Sync*	High	High	High	High	Low
Marketplace*	Medium	Medium	High	Medium	Medium
Shared team workspace*	Low	High	Medium	High	N/A
Audit logging*	N/A	Low	N/A	High	N/A
Approval workflows*	N/A	Medium	Low	High	N/A
SSO integration*	N/A	Low	N/A	High	N/A
Onboarding tour*	Medium	Low	N/A	N/A	High
Skill testing*	High	Medium	High	Medium	Low
Diff view*	Medium	Medium	High	Medium	N/A
Format conversion*	High	High	High	High	Medium

* Denotes features that are upcoming or proposed (not yet implemented in current SkillForge).